

Take-Home Assignment: Vendor Onboarding Tracker

The Business Problem

An Operations team is responsible for onboarding new vendors before they can start operating in a region.

Today, the team manages this process using a shared spreadsheet.

Current Process

A vendor goes through five stages:

- 1. Contract Sent**
A Sales PIC signs a new vendor. Once a deal is agreed, the Sales PIC adds a new row to the spreadsheet with the vendor's information.
 - 2. Contract Signed**
An Ops Coordinator follows up until the vendor signs the contract. The coordinator then changes the vendor's stage to Contract Signed and manually updates the "Last Updated" date.
 - 3. KYC Docs Received**
The vendor sends KYC documents, such as a business license and identification documents, by email. A coordinator updates the vendor's stage to KYC Docs Received.
 - 4. KYC Verified**
Compliance reviews the documents outside the spreadsheet, usually through Slack. If the documents pass, a coordinator updates the vendor to KYC Verified. If they fail, the vendor is asked to resend the required documents and remains at KYC Docs Received.
 - 5. Active**
Once KYC is verified, a coordinator performs a final activation step in another internal system and updates the vendor to Active.
-

Example Spreadsheet

Vendor	Region	Stage	Last Updated	Coordinator	Notes
Company A	HCMC	Active	2026-07-02	Linh	-
Company B	Can Tho	KYC Docs Received	2026-07-18	Huy	Waiting on business license re-upload
Company C	Hanoi	Contract Signed	2026-07-28	Linh	-
Company D	HCMC	KYC Verified	2026-08-01	Mai	-
Company E	Da Nang	KYC Docs Received	2026-07-15	Huy	-
Company F	HCMC	Contract Sent	2026-08-05	Mai	-

Three Ops Coordinators may work with the spreadsheet during the day.

The team has noticed that managing onboarding this way is becoming harder as the number of vendors increases. For example, vendors can sometimes remain at the same stage for a long time without anyone noticing, and it can be difficult to understand what happens when information in the spreadsheet looks incorrect.

Before You Build

Before implementing your solution, spend a little time thinking about the current spreadsheet-based process.

In your write-up, briefly describe:

- **2–3 problems** you see with the current process;
- how your solution addresses them;
- any assumptions you made.

There is no single correct answer. We are interested in understanding how you approach the problem as well as how you write code.

You **do not need to solve every problem** you identify.

What to Build

Build a small web application that replaces the spreadsheet for tracking vendors through these five stages:

- Contract Sent
- Contract Signed
- KYC Docs Received
- KYC Verified
- Active

A single-page application is enough.

Use whatever technology stack and approach you're comfortable with.

Requirements

1. Login

A coordinator should log in before using the application.

Authentication does **not** need to be production-ready. Mocked users, seeded accounts, or another simple login mechanism are completely acceptable.

We mainly want the application to know which coordinator is currently using it.

User registration, password reset, and user administration are out of scope.

2. View Vendors

Show a list of vendors and their current stages.

Display enough information for a coordinator to understand the current status of each vendor.

You may seed a handful of representative vendors. You do not need to reproduce the example spreadsheet exactly.

Creating and deleting vendors is out of scope.

3. Update a Vendor's Stage

A logged-in coordinator should be able to move a vendor to a different stage.

When this happens, the application should automatically record:

- who made the change;
- when the change happened;
- the previous stage;
- the new stage.

Coordinators should have a reasonable way to view this history.

4. Identify Stuck Vendors

Highlight vendors that have remained in their current stage for more than a defined number of days.

For example, you might decide that a vendor is considered "stuck" after 7 days without progressing to another stage.

The application should determine how long the vendor has been in its current stage rather than requiring the coordinator to manually enter a "Last Updated" date.

You may hardcode:

- the number of days before a vendor is considered stuck;
- the current date/time, if that makes the assignment easier to demonstrate or test.

Scope

Please keep the solution small.

You do **not** need to build:

- vendor creation or deletion;
- user registration or account management;
- production-grade authentication;
- email integration;
- the Compliance review system;

- the vendor activation system;
- notifications or reminders;
- real-time updates;
- production deployment infrastructure.

Seeded or mock data is acceptable.

If something in the workflow is unclear, make a reasonable assumption and mention it in your write-up.

Time Expectation

Please aim to spend **no more than 3 hours** on this assignment.

We don't expect a production-ready application or highly polished UI.

If you don't finish everything within the time, that's okay. Submit what you have and briefly explain:

- what you completed;
- what remains;
- what you would work on next.

We care more about your approach, code quality, and reasoning than the number of features completed.

What to Submit

Please submit the following.

1. Working Solution

- GitHub or GitLab repository;

2. Short Write-Up

A README or a few paragraphs is enough.

Please briefly cover:

- the 2–3 main problems you identified with the spreadsheet process;
- your overall approach;
- important technical decisions you made;
- assumptions you made;
- shortcuts or trade-offs you made because of the time limit;
- what you would improve if you had more time.

3. Tests

Include any tests you think are useful for your solution.

We do not expect a particular test coverage percentage.

We are more interested in **what you chose to test and why**.

What We Care About

We are not looking for the most complicated solution.

We will mainly look at:

- whether the application works correctly;
- whether you understood the business problem;
- whether your code is clear and reasonably structured;
- how you modeled the vendor workflow and history;
- how you handled basic edge cases;
- whether you can explain your decisions and trade-offs.

Visual design is not the main focus. A simple, usable interface is completely fine.

If you move forward to the next stage, we'll use your solution as the starting point for a technical discussion.

You may be asked how you would change or extend your solution if the requirements or scale changed, but you are **not expected to implement those scenarios as part of this assignment**.